Fake news detection in multiple platforms and languages

Pedro Henrique Arruda Faustini, Thiago Ferreira Covões

Federal University of ABC (UFABC)
Center of Mathematics, Computing, and Cognition
Avenida dos Estados 5001, 09210-580, Santo André (SP), Brazil

{pedro.faustini, thiago.covoes}@ufabc.edu.br

Abstract

The debate around fake news has grown recently because of the potential harm they can have on different fields, being politics one of the most affected. Due to the amount of news being published every day, several studies in computer science have proposed models using machine learning to detect fake news. However, most of these studies focus on news from one language (mostly English) or rely on characteristics of social media-specific platforms (like Twitter or Sina Weibo). Our work proposes to detect fake news using only text features that can be generated regardless of the source platform and are the most independent of the language as possible. We carried out experiments from five datasets, comprising both texts and social media posts, in three language groups: Germanic, Latin, and Slavic, and got competitive results when compared to benchmarks. We compared the results obtained through a custom set of features and with other popular techniques when dealing with natural language processing, such as bag-of-words and Word2Vec.

Keywords: Fake news, machine learning, supervised learning

*Corresponding author
Email address: `thiago.covoes@ufabc.edu.br` (Thiago Ferreira Covões)

1. Introduction

Historically, the press has the responsibility to publish facts of public interest. To do so, stories must pass through a series of journalistic criteria (White, 1950). The Internet, however, has a different structure that can disrupt this system. Anyone can fabricate content and spread it to the world. Social media is an example of a popular place where fake news spread, but they are not restricted to it. As a result, the need to identify fake news arises, regardless of where they are published and even in which language.

There has been a debate concerning the impact fake news can have in major events such as elections (Allcott & Gentzkow, 2017). Alongside the attention fake news has gathered in recent years, ways to detect them have motivated a wide range of works in different fields. For example, many websites dedicated to fact-checking rely on human labour to verify the authenticity of suspected news or claims. This approach has the advantage of focusing on news individually, but it is costly or even impractical at large scale considering the number of news that is published every day.

Furthermore, machine learning could be used as an ally in the task. Usually, works in the field train supervised learning models based on datasets of news that were manually annotated concerning their veracity. Then, the models infer whether unlabelled news are true or fake from extracted features from these datasets. This approach can handle a huge amount of data in a short time and can be a good start to raise alerts about suspicious texts.

Fake news detection is treated here as a classification problem under a supervised model. There are two phases (Han & Kamber, 2000). In the first one, a model is built from a training set. Each object in this set is labelled with a class $c_j \in C$, being $C = \{c_1, c_2, \ldots, c_l\}$ the set of l possible classes. In the current scenario, there are two classes: fake and true, and it is in this phase that a function is estimated. In the second phase, the estimated function is used to infer the label of unseen objects.

The main contribution of this paper is the evaluation, with the same method-

ology, of techniques for fake news detection in multiple platforms and languages. This is an opposed approach to what has commonly been done in the literature, in which proposed methods are tested only in a specific language and/or rely on specificities of digital platforms (usually the case of social networks) (Yang et al., 2012; Monteiro et al., 2018; Jin et al., 2016; Liu & Wu, 2018; Gravanis et al., 2019). As pointed by Zhou & Zafarani (2018), before the introduction of detection techniques to fake news, one has to answer some fundamental questions which are still unclear, such as how does fake news propagate from various domains or languages.

Here, we study the problem of fake news detection in three different languages, all of them from distinct origins: English is a Germanic language, and it is a standard choice for many natural language processing studies. On the other hand, we find fewer works on Portuguese, a Latin language, and Bulgarian, Slavic. Fake news is not restricted to any specific language or country, therefore a more generic approach for detecting them is salutary. We compared four distinct text feature sets, all of them they can be generated regardless of the source platform, i.e., they do not rely on specificities like a particular metadata from a given social network, or time information that might not be available in a website text, for example.

The remainder of this paper is organised as follows. Section 2 discusses related work. Afterwards, an overview of document representation techniques is presented in Section 3. The features we used in this work, as well as the characteristics of the datasets, are presented in Section 4. Section 5 discuss precautions to avoid data leakage when training models, with special attention when dealing with tweets. Then, experimental results for fake news detection are discussed in Section 6. Finally, Section 7 concludes the paper and points to future work.

2. Related Work

Studies on fake news detection regularly take data either from social media like Twitter and Sina Weibo or texts that are extracted from websites. In many times, studies whose source platforms are social media also rely on specific properties of those platforms (like the associated metadata to track their dissemination).

Gupta et al. (2012) used supervised learning to initialise credibility values in a propagation network. Then, a credibility propagation system evaluated a topic as true or not. They got an accuracy as high as 86.8%. Ruchansky et al. (2017) proposed a model employing deep learning that combines the text itself, the user response it receives and the source users who are promoting the article to detect fake news. First, a Recurrent Neural Network captures temporal patterns of users' activities about a text, followed by another module that learns the source characteristic in the users' behaviours. Another model based on the propagation of messages was studied by Liu & Wu (2018) using recurrent and convolutional neural networks. They build propagation paths based on users' characteristics as soon as messages start to spread. Similarly, Wu & Liu (2018) analysed how traces of information diffusion can be exploited to label a message, based on who forwarded it and when. Other deep learning approaches have been investigated, such as exploring temporal aspects of fake news (Yu et al., 2019) and emotional signals on text (Giachanou et al., 2019).

In another work, researchers explored conflicting information about a topic Jin et al. (2016). They identify different viewpoints and build a credibility propagation network of posts from Sina Weibo platform, linked according to their relations, either opposing or supporting. Finally, a credibility propagation system classifies the event, with an accuracy of 84%. Yang et al. (2012) collected a dataset and labelled data according to a platform's official service for rumours. Then they studied the effect of 19 features for classification, including features specific to Sina Weibo platform, and got an accuracy of 78.6%. Many of those features were presented by Castillo et al. (2011), which focus on inferring credi-

bility of tweets but also proposed new ones, that exploit characteristics of Sina Weibo platform.

Typically, approaches like these are dependent not just of the news themselves, but also in the environment they are spread. For one side, authors can gather more information to help classification. On the other hand, such information may be only available for that specific platform, and hence the method may not generalise well, or even works, for others.

There are works focusing on text rather than environment features, though. Ahmed et al. (2017) studied how different n-grams lengths impact on fake news detection. They trained six classifiers with fake and true content using bag-of-words with tf and tf-idf and got an accuracy of 92%. Other text approaches have been adopted in the literature. Five datasets were gathered into one by Gravanis et al. (2019) and split in random batches. Authors evaluated different feature sets and word embeddings, achieving an accuracy as high as 95%. Despite the effort to build a model from different datasets, they all contained data in English language. Monteiro et al. (2018) built a Portuguese dataset of true and fake news on various subjects and extracted features based on linguistic properties. They trained a Support Vector Machine (SVM) (Cortes & Vapnik, 1995) with different sets of features (e.g., bag-of-words or customised features) and got 89% of accuracy. In the work of Hardalov M. (2018), natural language processing was also adopted to identify fake news in a language other than English. This time, data from Bulgarian sources were collected, and they measured how capitalisation, punctuation, sentiment polarity and other features helped to detect fake news.

Helmstetter & Paulheim (2018) used the source of the news, whether they were considered trustworthy or not, to label the news, instead of labelling objects individually (hence the name "weakly supervised learning"). Many times, fake news share characteristics of satirical content. In this way, Horne & Adali (2017) claim that fake news are also more similar to satirical news than to true news. In their study, they extract text features, as well as sentiment analysis, to distinguish the classes. The study of Bhattacharjee et al. (2017) proposed a

human-machine collaborative approach to evaluating news veracity. They start with a small number of labelled objects, and the model is gradually updated to improve performance.

The methods described in these last works could, in theory, be applied to news from different languages and platforms. However, in practice, usually they are just evaluated in one language and one platform (just websites or just social media). Therefore, we evaluate these techniques in a wider range of domains, since fake news dissemination is not restricted to one given language or platform. Our analysis seeks to validate that the same methodology may be applied to different settings (concerning language and platform). Also, as fact-checking is a manual and slow process, collecting a large set of fake news is, usually, not possible. Due to this, we do not employ Deep Learning techniques, and focus on others well known machine learning algorithms.

3. Background

Frequently, the complexity of a classifier depends on the number of inputs it receives, either for the space or time it will require (Alpaydin, 2010). Using bag-of-words tends to imply in large matrices to be processed, which leads to the problem of the curse of dimensionality (Bellman, 1957). It also does not take into account the semantic relations about words. These issues emphasise the need for a judicious choice of document representation.

About the first issue, the two main groups of techniques to perform dimensionality reduction are feature selection and feature extraction (Tommasel & Godoy, 2018). The first one selects a subset of features (for example, in a bag-of-words approach, it would only select a subset of words). The second one generates a new set of features, customarily smaller than the original set. About the second one, word embeddings techniques aim to convert words to numerical vectors usually holding special semantic properties (Jurafsky & Martin, 2000).

In this paper we use two document representation algorithms for text mining: DCDistance (Ferreira et al., 2018) and Word2Vec (Mikolov et al., 2013b)(Mikolov

et al., 2013a). The first one is a dimensionality reduction algorithm that reduces the number of features down to the number of classes. The second one is a document representation algorithm that maps each word to a vector of a given size. More details of each algorithm are given below.

3.1. DCDistance

Document-Class Distance (DCDistance) algorithm works as follows: it receives the original (supposedly big) feature matrix. Feature vectors of objects from the same class are summed. Hence, if there are c classes in the dataset, there will be generated c representative vectors, one for each class. Last, each object will be represented by c features. Each feature is the distance between itself and a representative vector (Ferreira et al., 2018). Algorithm 1 presents the main steps of the algorithm, where L is the number of classes and N is the number of objects. Figure 1 summarises this process in a pedagogical example.

When dealing with text, a popular distance metric is the cosine distance. Let $\vec{v}$ and $\vec{w}$ be vectors that represent two text documents, and t their size (it is required they both have the same size). Cosine distance is calculate as follows:

$$cos_dst(\vec{v}\ ,\vec{w}\ ) = 1 - \left| \frac{\sum_{i=1}^{t} v_i w_i}{\sqrt{\sum_{i=1}^{t} v_i^2}\sqrt{\sum_{i=1}^{t} w_i^2}} \right| . \tag{1}$$

The concept behind the cosine distance is to measure how big is the angle between two vectors, regardless of their magnitude. The higher the cosine distance is, the bigger the distance between the two vectors is as well. This is convenient for text mining because different vectors representing text documents often have very discrepant magnitudes.

Original object	**Class**
[1, 0, 0, 1, 0]	A
[0, 1, 1, 0, 0]	A
[1, 0, 0, 1, 0]	A
[0, 1, 0, 0, 1]	B
[1, 1, 0, 1, 0]	B

$\xrightarrow{\text{Class vectors}}$ CV_A: [2 1 1 2 0] CV_B: [1 2 1 1 1] $\xrightarrow{\text{Distances from objects to class vectors}}$

Reduced representation	**Class**
[0.10, 0.47]	A
[0.55, 0.47]	A
[0.11, 0.47]	A
[0.78, 0.20]	B
[0.09, 0.13]	B

Figure 1: Example of an execution of DCDistance

Algorithm 1 DCDistance

Input: dataset $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^{N}$, labels $\mathcal{Y} = \{y_i\}_{i=1}^{N}$

Output: reduced dataset $\mathcal{X}$'

Let $\mathcal{C} = \{\mathbf{c}_l\}_{l=1}^{L}$ be the representatives of each class (summed vectors)
$\mathcal{X}' = \emptyset$
for each $\mathbf{x}_i \in \mathcal{X}$ **do**
    Let $\mathbf{x}'_i$ be the transformed vector $\mathbf{x}_i$
    for each $\mathbf{c}_l \in \mathcal{C}$ **do**
        $x'_{il} = distance(\mathbf{x}_i, \mathbf{c}_l)$
    end for
    $\mathcal{X}' = \mathcal{X}' \cup \{\mathbf{x}'_i\}$
end for

A reasonable scenario to apply DCDistance is to reduce the size of a bag-of-words matrix, for example. Such matrices are often big, and if the number of classes in the problem is small (what typically is the case), the algorithm can reduce drastically the size of dimensionality.

3.2. Word2Vec

The goal of Word2Vec is to represent each word as a numerical vector (Mikolov et al., 2013b). Moreover, words like *computer* and *machine* would have similar vectors when compared to the word vector of *rice*, for example.

The algorithm uses a neural network with an input, a hidden and an output layer. There are two variations of this algorithm: CBOW and Skip-Gram. The first receives several words, called context, which has a fixed size determined *a priori*. It outputs one word, called the target word. The second variation does the reverse: it receives a word and outputs the context for that word. Figure 2 depicts both models.

Regardless of the variation, words in input and output layers are each one represented as one-hot encoded vectors. It means that if the vocabulary has V different words, each word is represented by a $1xV$ vector, will all entries set to

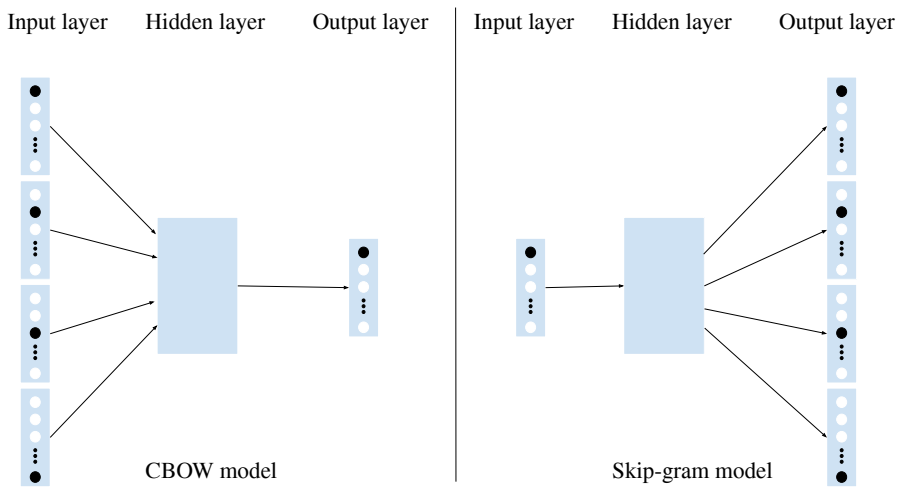


Figure 2: Variations of Word2Vec

0 except one, representing that word. Hence, each word is uniquely identified.

To output either the context or target word according to the model, a neural network is trained with weight vectors of size determined by a parameter. After training is done, the weight vectors between the input and hidden layer are the word vectors.

One of the reasons for the popularity of Word2Vec is the fact that the context of the words is taken into account for word vectors creation, something that is irrelevant in bag-of-words. Word2Vec holds semantic relations amongst words, something that approaches like bag-of-words also fail to do.

However, document classification tasks commonly require to reduce words not to vectors, but numbers. This process is natural in bag-of-words, but since Word2Vec maps words to vectors, aggregation must be done to unify all word vectors from a document into a single one. One straightforward strategy is to sum all vectors (and potentially divide the resulting vector by the number of words the document has).

4. Features and datasets

Beyond natural language representation, we also trained models with a customised set of features. This set is based on Faustini & Covões (2019) with few editions. The concept of these features is that they can be extracted from raw texts of news, being weakly dependent on the language.

Table 1 lists the features. Features 1 to 8 can be easily extracted from raw texts. For features 9 to 12, we used Polyglot[1]. It was proposed by Al-Rfou et al. (2013), in which authors address the problem that natural language processing systems rely heavily on English features, which makes such systems hard to port to other languages. Therefore, they generated word embeddings for several languages and made their pre-trained models available. The training was made over Wikipedia pages. Tools in the library include part-of-speech tagging and sentiment analysis, that we used for feature generation.

Specifically for feature 13, in order to reduce the word vectors from a matrix to a single value, for each text we got its word vectors, then summed them (column-wise), resulting in one vector. This feature is the mean of this last vector. All Word2Vec models[2] we used have a vector size of 100 with a window size of 10 words, trained using the CBOW approach. They were made available by Fares et al. (2017).

Finally, for feature 14 we used the Hunspell[3] spell checker.

For evaluation, we used five datasets: FakeBrCorpus (Monteiro et al., 2018), TwitterBR (Faustini & Covões, 2019), Fake_or_real_news Bhattacharjee et al. (2017), Fakenewsdata1 (Horne & Adali, 2017), and btvlifestyle (Hardalov M., 2018). They are from three different languages, coming from distinct groups: Portuguese (Latin), English (Germanic) and Bulgarian (Slavic). The source of their material are either social media (Twitter) or websites. Table 2 shows statistics of each dataset.

[1] https://github.com/aboSamoor/polyglot
[2] http://vectors.nlpl.eu/repository/
[3] https://github.com/blatinier/pyhunspell

Table 1: Textual features extracted from news content

Id	Feature	Id	Feature
1	Proportion of uppercase characters	8	Words per sentence
2	Proportion of exclamation marks	9	Proportion of adjectives
3	Proportion of question marks	10	Proportion of adverbs
4	Text has exclamation marks	11	Proportion of nouns
5	Number of unique words	12	Sentiment of message
6	Number of sentences	13	Word2Vec representation
7	Number of characters	14	Spelling errors

FakeBrCorpus is available from Monteiro et al. (2018). It has 7,200 texts from websites, all of them in Portuguese. These texts are from different subjects, being politics the most common one.

TwitterBR is a dataset of Brazilian tweets made available in Faustini & Covões (2019). One difference between FakeBrCorpus and this dataset, beyond the source of the material (websites in the first and social media in the second) is that TwitterBR is focused on politics, as opposed to FakeBrCorpus, that brings news from different subjects.

Fake_or_real_news has thousands of articles labelled as false or fake. It was made public by KDNuggets, and it has been used by academic papers (Bhattacharjee et al., 2017). We only selected those texts with more than 280 characters.

Fakenewsdata1 is a dataset made available in Horne & Adali (2017). It has fake, true and satirical stories about politics in English. We only used content labelled as fake or true, ignoring the satirical class.

Last, btvlifestyle was made available from Hardalov M. (2018). Authors studied natural language processing techniques to identify fake news and provided a dataset they collected. It is in Bulgarian language and texts are from websites.

Table 2: Datasets employed in the experiments

Dataset	Fake news	True news	Total
TwitterBR	4,392	4,589	8,981
FakeBrCorpus	3,600	3,600	7,200
FakeNewsData1	75	75	150
FakeOrRealNews	2,962	2,941	5,903
btvlifestyle	69	68	137

5. Methodology

One major concern when evaluating models is to avoid that training data be used during testing, usually known as data leakage (Abu-Mostafa et al., 2012). One of the most common examples is to fit a model with some object, and during test phase using that same model for classification. Another trivial example is to scale the whole dataset, before splitting it into train and test sets. In these both examples, classification performance can be artificially improved because the model ends up using for classification data it has already been exposed during training somehow.

In machine learning experimental settings, it is common practice to adopt k-fold cross-validation (Tan et al., 2005). However, even though in each run a different model is fitted, cross-validation *per se* does not guarantee that data leakage does not occur in fake news settings.

For example, when classifying tweets, it is common to have several tweets of the same topic. Therefore, they can be very similar to each other (as a retweet). In k-fold cross-validation, it is possible that tweets from the same topic end up in different folds because the number of folds is fixed, whilst the number of tweets per topic is not. Eventually, one fold will be selected for testing, but the model will be fitted using data from tweets that are in other folds, but are very similar to those it will have to classify.

To avoid this kind of data leakage, we took an additional care when dealing with tweets (TwitterBR dataset). Each fold has only tweets of the same topic.

As we have 108 topics, we end up with 108 folds. In each iteration, one fold is selected as a testing fold, as well as one other fold with a different label. This other chosen folder is the one with the closest number of instances, so we try to keep tests balanced. The remaining ones are used for training. We ensure each fold is used at least once for testing. This leave-topic-out approach has the drawback that not all folds have the same size but, on the other hand, it brings the greater benefit of avoiding data leakage. Finally, as fake news detection is more important when the news is recently spread, this methodology provides a more realistic scenario.

We conducted, in each dataset, four sets of tests: one using the custom features described in Section 4, other two using Word2Vec and DCDistance, described in Section 3 and finally another using bag-of-words with tf-idf.

In each set, four algorithms were used: KNN, Random Forest Breiman (2001), Gaussian Naïve Bayes (Multinomial for bag-of-words) and SVM Cortes & Vapnik (1995). Code was written with scikit-learn (0.20.3) (Pedregosa et al., 2011) in Python (3.7.3). We set, whenever necessary, the seed parameter to 42. We used 5-fold cross-validation. Results were obtained after grid-search with the following structure: Naïve Bayes (NB) has no hyperparameters to tune. For KNN, we tested 1, 3, 5 and 7 neighbours. For SVM, we tested kernels sigmoid, linear, radial-basis function and polynomial with $\gamma = \frac{1}{M}$, where M is the number of features. For Random Forest (RF), we tested different numbers of trees, ranging from 1 to 1000, with intervals of 50 trees. Other values were scikit-learn defaults.

6. Experiments

As explained in Section 4, we converted words from datasets to vectors using the models provided from Fares et al. (2017). Table 3 shows the fraction of words in each text, on average, that was not possible to find a correspondent word vector. As one would expect, TwitterBR is the dataset with the biggest percentage of missing words, since Twitter is a social media platform and users

frequently write slangs or even misspelt words.

Table 3: Failure rates to convert words to vectors

Dataset	Missing words
TwitterBR	7.55%
FakeBrCorpus	1.86%
FakeNewsData1	1.26%
FakeOrRealNews	1.74%
btvlifestyle	2.19%

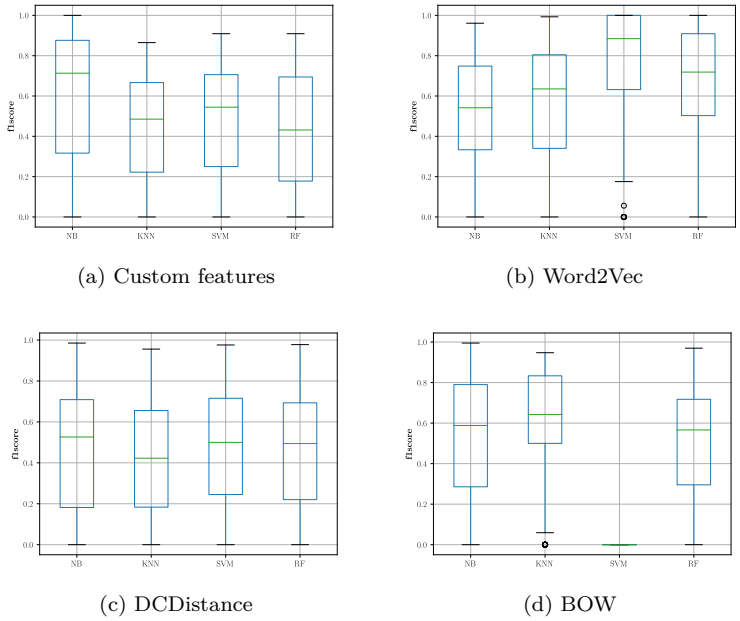
We also compared the effect of doing conventional 5-fold cross-validation (hereafter referred to as TwitterBr) and cross-validation with two topics per fold in the Twitter dataset (hereafter referred to as TwitterBR LTO), as explained in Section 5. For this comparison, algorithms in TwitterBR entries were run with the same parameters selected as the best in TwitterBR LTO tests. Therefore, we ensure we only measure the effect of changing the way tweets are organised within folds. Table 4 shows F1-Score results and Table 5 shows accuracy scores.

With the LTO approach, we notice a higher standard deviation in results — almost all of them in two digits space. This is expected, since the number of tweets in each topic differs. The 8,981 tweets are spread over 108 topics. It gives a mean of 83 tweets per topic. However, there is a high standard deviation in this average, of $\pm$ 167 tweets. Each wrong or right classification in a topic with few objects can have a high impact on results, and thus standard deviation tends to be higher than when the evaluation is done with balanced folds - but with the risk of data leakage.

By looking at the accuracy entries in Table 5, we see that Twitter LTO entries show a numerically higher accuracy than TwitterBR entries in 8 out of 16 cases, but with little higher standard deviation in general. Comparison for F1-Score is problematic due to much higher standard deviations in LTO. In Figure 3 we show the the deviation across F1-Score measures. The high deviation can be explained because the folds have different sizes, hence leading

Table 4: F1-Score results for Naive Bayes, K-Nearest Neighbours, Support Vector Machines and Random Forest (best ones in bold)

Dataset	Feature set	NB	KNN	SVM	RF
FakeBrCorpus	Customised	50% (±7)	71% (±3)	73% (±4)	74% (±4)
	Word2Vec	69% (±3)	76% (±1)	84% (±3)	77% (±3)
	DCD	62% (±2)	80% (±2)	81% (±1)	80% (±2)
	BOW	85% (±2)	73% (±4)	**91%** (±2)	88% (±2)
FakeNewsData1	Customised	33% (±23)	73% (±5)	79% (±4)	78% (±5)
	Word2Vec	70% (±10)	77% (±10)	78% (±8)	77% (±11)
	DCD	54% (±14)	80% (±5)	56% (±17)	66% (±8)
	BOW	69% (±8)	64% (±13)	83% (±5)	**86%** (±6)
FakeOrRealNews	Customised	40% (±7)	67% (±1)	74% (±1)	76% (±1)
	Word2Vec	68% (±2)	82% (±1)	89% (±1)	84% (±1)
	DCD	63% (±1)	85% (±1)	78% (±1)	84% (±1)
	BOW	76% (±1)	79% (±2)	**94%** (±1)	90% (±0)
btvlifestyle	Customised	78% (±8)	80% (±5)	83% (±5)	83% (±7)
	Word2Vec	91% (±2)	90% (±3)	94% (±3)	93% (±4)
	DCD	68% (±20)	92% (±5)	72% (±2)	82% (±8)
	BOW	91% (±6)	94% (±5)	93% (±4)	**95%** (±2)
TwitterBR	Customised	49% (±18)	63% (±14)	60% (±14)	62% (±18)
	Word2Vec	45% (±21)	67% (±19)	69% (±17)	71% (±16)
	DCD	71% (±20)	**79%** (±12)	**79%** (±11)	77% (±13)
	BOW	73% (±19)	77% (±10)	0% (±0)	64% (±19)
TwitterBR LTO	Customised	59% (±34)	43% (±26)	47% (±29)	44% (±28)
	Word2Vec	51% (±30)	55% (±31)	**76%** (±29)	68% (±30)
	DCD	48% (±30)	43% (±26)	48% (±29)	46% (±28)
	BOW	54% (±30)	59% (±29)	0% (±0)	48% (±28)



(a) Custom features (b) Word2Vec

(c) DCDistance (d) BOW

Figure 3: Boxplots for F1-Scores measures in Twitter dataset (LTO approach)

to imbalanced test data.

Investigating SVM's results for TwitterBR dataset further, it classified all instances as true class, obtaining a high accuracy. However, this lead to a precision for the false class of 0% — and thus the same with the F1-Score.

In general, Random Forest and SVM are the two algorithms that achieved the best results. The prevalence of bag-of-words is not unseen. In Monteiro et al. (2018), authors achieved their best results also using 5-fold cross-validation and either bag-of-words, or bag-of-words with other features, in FakeBrCorpus dataset (88% and 89% accuracy respectively).

With regard to FakeNewsData1 dataset, the best result in its benchmark is an accuracy of 78%, using a customised set of features and 5-fold cross-validation (Horne & Adali, 2017). Our best result is numerically slightly better when also using our customised set of features (79%). About FakeOrRealNews dataset, there is a benchmark result of an accuracy as high as 92.7% (Bhattacharjee

et al., 2017), even though researchers used a different methodology for evaluation that limits precise comparisons. In this dataset, bag-of-words got an accuracy of 94%. Benchmark for btvlifestyle dataset presents a maximum accuracy of 76% (Hardalov M., 2018). They do not mention using cross-validation, hence comparisons with our experiments, that in principle achieved significantly better results, is also limited. Finally, the dataset from Faustini & Covões (2019) was only assessed in a one-class classification scenario previous to this work.

Table 6 shows the best parameters after grid-search. They were achieved based on an accuracy score. F1-scores of Table 4 were the ones achieved with these same parameters. We notice that the radial-basis function, polynomial and linear kernels as the most common ones, far more common than sigmoid kernel. About the number of neighbours in KNN, we see that seven is the most prevalent one, especially in bigger datasets. Finally, the Random Forest showed a very distinct number of trees along the experiments, with a median value of 426 trees.

We conducted a Friedman statistical test (Demšar, 2006) with the null hypothesis being that the F1-Score results do not significantly change according to the feature set chosen, i.e., they all come from the same distribution. We have $N = 24$ (24 pairs <dataset, algorithm>) and $k = 4$ (four different feature sets). We got $\mathcal{X}_f^2 = 19.02$ and $F_f = 8.26$, according to the F distribution with 4-1 = 3 and (4-1) $\times$ (24-1) = 69 degrees of freedom. The critical value for F(3, 69) = 2.74 for $\alpha = 0.05$. Since our measure is 8.26 we reject the null hypothesis.

We then conduct a Nemenyi posthoc test. The corresponding critical difference is 0.96. We can see that the performance of the custom features are significantly worse than the bag-of-words and Word2Vec, but the same can not be said about DCDistance, which presented no significant differences to BOW and Word2Vec, as it can be seen in Figure 4. Table 7 show the mean rank for each feature set.

From this statistical test, an interesting finding is that results with DCDistance were not statistically significantly worse than those with bag-of-words, despite the huge dimensionality reduction that DCDistance performs.

Table 5: Accuracy results for Naive Bayes, K-Nearest Neighbours, Support Vector Machines and Random Forest (best ones in bold)

Dataset	Feature set	NB	KNN	SVM	RF
FakeBrCorpus	Customised	64% (±3)	71% (±3)	74% (±2)	74% (±2)
	Word2Vec	70% (±2)	77% (±2)	84% (±2)	79% (±2)
	DCD	60% (±1)	81% (±2)	80% (±1)	80% (±1)
	BOW	85% (±1)	75% (±3)	**91%** (±1)	88% (±1)
FakeNewsData1	Customised	59% (±8)	75% (±5)	79% (±4)	78% (±6)
	Word2Vec	65% (±12)	77% (±11)	77% (±8)	77% (±10)
	DCD	63% (±10)	81% (±4)	65% (±11)	68% (±6)
	BOW	75% (±6)	69% (±10)	83% (±4)	**86%** (±5)
FakeOrRealNews	Customised	61% (±2)	70% (±1)	75% (±1)	76% (±1)
	Word2Vec	69% (±1)	82% (±1)	88% (±1)	84% (±0)
	DCD	67% (±0)	85% (±1)	77% (±1)	84% (±1)
	BOW	80% (±1)	81% (±1)	**94%** (±1)	90% (±1)
btvlifestyle	Customised	71% (±11)	78% (±6)	82% (±7)	82% (±7)
	Word2Vec	91% (±3)	89% (±3)	94% (±4)	93% (±4)
	DCD	74% (±12)	92% (±5)	61% (±4)	80% (±8)
	BOW	89% (±7)	93% (±5)	92% (±4)	**95%** (±2)
TwitterBR	Customised	60% (±12)	69% (±9)	66% (±9)	70% (±9)
	Word2Vec	58% (±12)	74% (±11)	75% (±10)	77% (±9)
	DCD	76% (±12)	**81%** (±9)	80% (±8)	80% (±9)
	BOW	80% (±11)	79% (±9)	51% (±0)	67% (±11)
TwitterBR LTO	Customised	78% (±19)	69% (±14)	73% (±17)	71% (±15)
	Word2Vec	72% (±16)	77% (±17)	**84%** (±21)	82% (±17)
	DCD	61% (±18)	48% (±20)	45% (±28)	49% (±23)
	BOW	71% (±19)	76% (±19)	64% (±17)	55% (±23)

Table 6: Best parameters found after grid-search

Dataset	**Feature set**	**KNN**	**SVM**	**RF**
FakeBrCorpus	Customised	k: 7	kernel: *rbf*	estimators: 601
	Word2Vec	k: 7	kernel: *linear*	estimators: 951
	DCD	k: 7	kernel: *linear*	estimators: 501
	BOW	k: 7	kernel: *linear*	estimators: 751
FakeNewsData1	Customised	k: 3	kernel: *rbf*	estimators: 651
	Word2Vec	k: 7	kernel: *rbf*	estimators: 51
	DCD	k: 7	kernel: *rbf*	estimators: 451
	BOW	k: 5	kernel: *linear*	estimators: 251
FakeOrRealNews	Customised	k: 7	kernel: *rbf*	estimators: 851
	Word2Vec	k: 7	kernel: *rbf*	estimators: 401
	DCD	k: 7	kernel: *linear*	estimators: 251
	BOW	k: 1	kernel: *linear*	estimators: 901
btvlifestyle	Customised	k: 5	kernel: *rbf*	estimators: 101
	Word2Vec	k: 1	kernel: *rbf*	estimators: 51
	DCD	k: 3	kernel: *rbf*	estimators: 1
	BOW	k: 5	kernel: *linear*	estimators: 951
TwitterBR	Customised	k: 3	kernel: *polynomial*	estimators: 201
	Word2Vec	k: 7	kernel: *polynomial*	estimators: 951
	DCD	k: 1	kernel: *polynomial*	estimators: 1
	BOW	k: 5	kernel: *polynomial*	estimators: 1

Table 7: Average ranks for each set of features

Feature set	Average rank
Custom features	3.396
Word2Vec	2.187
DCDistance	2.562
BOW	1.854

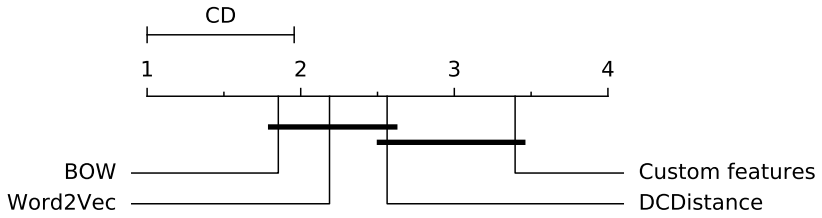
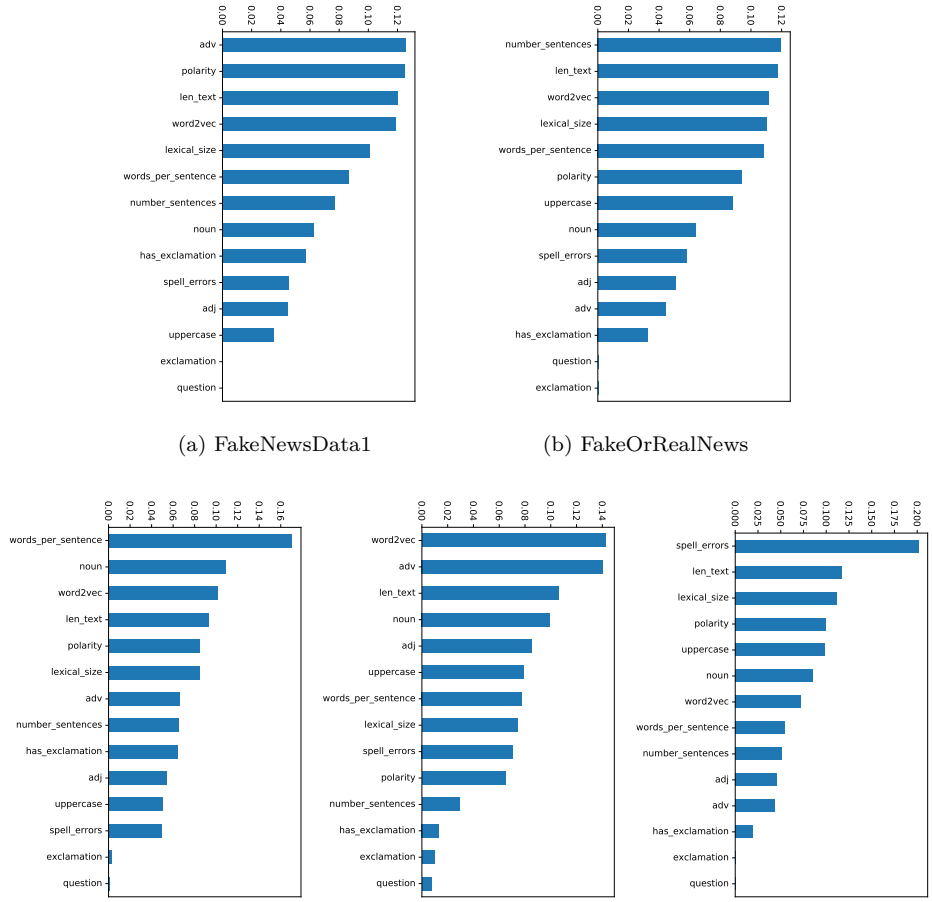


Figure 4: Differences in methods' performances

By analysing the results, we noticed that Support Vector Machines and Random Forest outperformed other classification algorithms either in accuracy or F1-Score measures. About the feature set, bag-of-words approach achieved the best results. It is common, when dealing with text, that matrices generated by such approach end up being large. In this sense, DCDistance showed a useful algorithm to reduce dimensionality without losing too much performance.

Figure 5 shows the importance of each customised feature in each dataset. Feature importances were measured according to the Gini impurity (Breiman et al., 1984) with respect to the Random Forest classifier. For each dataset, the number of estimators is the one returned as the best result after grid-search.

We see some general trends amongst the features. The proportion of exclamation and question marks in texts seems to offer little help in all datasets. The opposite can be said about the length of the text and Word2Vec features. The lexical size (unique words) and the sentiment (polarity) are helpful in all datasets, with exception of Twitter. This may be explained by the differences in writing styles that social media have.



(a) FakeNewsData1 (b) FakeOrRealNews

(c) FakeBrCorpus (d) TwitterBR (e) btv-lifestyle

Figure 5: Feature importances in each datasets

7. Conclusions

In this paper, we propose to detect fake news using only text features which are the most independent of the language as possible. Moreover, they can be generated regardless of the source platform.

We noticed some general trends amongst the features. Whilst the proportion of exclamation and question marks in texts seems to offer little help, the opposite happens with the length of the text and Word2Vec representation features. The lexical size (unique words) and the sentiment (polarity) are helpful in all datasets, except for Twitter, according to Gini impurity.

From the experiments, we noticed that Support Vector Machines and Random Forest outperformed other classification algorithms. About the feature set, bag-of-words approach achieved the best results in general in our experiments — even though not statistically significantly better than all other feature sets. From this, another interesting finding is that results with DCDistance were not statistically significantly worse than those with bag-of-words, despite the huge dimensionality reduction that DCDistance performs.

The topic of fake news is a debate in many fields, and the computer science literature has usually relied on studies focusing on one kind of platform or language. Experiments in this work were conducted in languages from different groups (Latin, Germanic and Slavic). Moreover, datasets are originated from different platforms, either websites or social media.

8. Acknowledgements

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001. This study was also financed in part by UFABC.

References

Abu-Mostafa, Y. S., Magdon-Ismail, M., & Lin, H.-T. (2012). *Learning From Data*. AMLBook.

Ahmed, H., Traore, I., & Saad, S. (2017). Detecting opinion spams and fake news using text classification. *Security and Privacy*, *1*, e9. doi:10.1002/spy2.9.

Al-Rfou, R., Perozzi, B., & Skiena, S. (2013). Polyglot: Distributed word representations for multilingual nlp. In *Proceedings of the Seventeenth Conference on Computational Natural Language Learning* (pp. 183–192). Sofia, Bulgaria: Association for Computational Linguistics. URL: http://www.aclweb.org/anthology/W13-3520.

Allcott, H., & Gentzkow, M. (2017). Social media and fake news in the 2016 election. *Journal of Economic Perspectives*, .

Alpaydin, E. (2010). *Introduction to Machine Learning.* (2nd ed.). The MIT Press.

Bellman, R. (1957). *Dynamic Programming.* (1st ed.). Princeton, NJ, USA: Princeton University Press.

Bhattacharjee, S. D., Talukder, A., & Balantrapu, B. V. (2017). Active learning based news veracity detection with feature weighting and deep-shallow fusion. In *2017 IEEE International Conference on Big Data (Big Data)* (pp. 556–565). doi:10.1109/BigData.2017.8257971.

Breiman, L. (2001). Random Forests. *Mach. Learn.*, *45*, 5–32. doi:10.1023/A:1010933404324.

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees.* The Wadsworth statistics/probability series. Monterey, CA: Wadsworth & Brooks/Cole Advanced Books & Software.

Castillo, C., Mendoza, M., & Poblete, B. (2011). Information Credibility on Twitter. In *Proceedings of the 20th International Conference on World Wide Web* WWW '11 (pp. 675–684). New York, NY, USA: ACM. doi:10.1145/1963405.1963500.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, *20*, 273–297. doi:10.1023/A:1022627411411.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.*, *7*, 1–30. URL: http://dl.acm.org/citation.cfm?id=1248547.1248548.

Fares, M., Kutuzov, A., Oepen, S., & Velldal, E. (2017). Word vectors, reuse, and replicability: Towards a community repository of large-text resources. In *Proceedings of the 21st Nordic Conference on Computational Linguistics* (pp. 271–276). Gothenburg, Sweden: Association for Computational Linguistics. URL: https://www.aclweb.org/anthology/W17-0237.

Faustini, P., & Covões, T. (2019). Fake news detection using one-class classification. In *2019 8th Brazilian Conference on Intelligent Systems (BRACIS)* (pp. 592–597). doi:10.1109/BRACIS.2019.00109.

Ferreira, C. H. P., de Medeiros, D. M. R., & de França, F. O. (2018). DCDistance: A Supervised Text Document Feature extraction based on class labels. *CoRR*, *abs/1801.04554*. arXiv:1801.04554.

Giachanou, A., Rosso, P., & Crestani, F. (2019). Leveraging emotional signals for credibility detection. In *Proceedings of the 42Nd International ACM SIGIR Conference on Research and Development in Information Retrieval* SIGIR'19 (pp. 877–880). New York, NY, USA: ACM. doi:10.1145/3331184.3331285.

Gravanis, G., Vakali, A., Diamantaras, K., & Karadais, P. (2019). Behind the cues: A benchmarking study for fake news detection. *Expert Systems with Applications*, *128*, 201 – 213. URL: http://www.sciencedirect.com/science/article/pii/S0957417419301988. doi:https://doi.org/10.1016/j.eswa.2019.03.036.

Gupta, M., Zhao, P., & Han, J. (2012). Evaluating Event Credibility on Twitter. In *SDM* (pp. 153–164). SIAM / Omnipress.

Han, J., & Kamber, M. (2000). *Data Mining: Concepts and Techniques*. Morgan Kaufmann.

Hardalov M., N. P., Koychev I. (2018). In search of credible news. In *Artificial Intelligence: Methodology, Systems, and Applications* AIMSA (pp. 172–180). Springer, Cham. doi:10.1007/978-3-319-44748-3_17.

Helmstetter, S., & Paulheim, H. (2018). Weakly Supervised Learning for Fake News Detection on Twitter. In *2018 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)* (pp. 274–277). doi:10.1109/ASONAM.2018.8508520.

Horne, B. D., & Adali, S. (2017). This Just In: Fake News Packs a Lot in Title, Uses Simpler, Repetitive Content in Text Body, More Similar to Satire than Real News. *CoRR*, *abs/1703.09398*.

Jin, Z., Cao, J., Zhang, Y., & Luo, J. (2016). News verification by exploiting conflicting social viewpoints in microblogs. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence* AAAI'16 (pp. 2972–2978). AAAI Press.

Jurafsky, D., & Martin, J. H. (2000). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. (1st ed.). Upper Saddle River, NJ, USA: Prentice Hall PTR.

Liu, Y., & Wu, Y. B. (2018). Early detection of fake news on social media through propagation path classification with recurrent and convolutional networks. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018* (pp. 354–361). URL: https://www.aaai.org/ocs/index.php/AAAI/AAAI18/paper/view/16826.

Mikolov, T., Chen, K., Corrado, G. S., & Dean, J. (2013a). Efficient estimation of word representations in vector space. URL: http://arxiv.org/abs/1301.3781.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013b). Distributed representations of words and phrases and their compositionality. *CoRR*, *abs/1310.4546*. URL: http://arxiv.org/abs/1310.4546. arXiv:1310.4546.

Monteiro, R. A., Santos, R. L. S., Pardo, T. A. S., de Almeida, T. A., Ruiz, E. E. S., & Vale, O. A. (2018). Contributions to the study of fake news in portuguese: New corpus and automatic detection results. In *Computational Processing of the Portuguese Language* (pp. 324–334). Springer International Publishing. doi:10.1007/978-3-319-99722-3_33.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python . *Journal of Machine Learning Research*, *12*, 2825–2830.

Ruchansky, N., Seo, S., & Liu, Y. (2017). CSI: A Hybrid Deep Model for Fake News Detection. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management* CIKM '17 (pp. 797–806). New York, NY, USA: ACM. doi:10.1145/3132847.3132877.

Tan, P.-N., Steinbach, M., & Kumar, V. (2005). *Introduction to Data Mining, (First Edition)*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.

Tommasel, A., & Godoy, D. (2018). Short-text feature construction and selection in social media data: a survey. *Artificial Intelligence Review*, *49*, 301–338. doi:10.1007/s10462-016-9528-0.

White, D. M. (1950). The "Gate Keeper": A Case Study in the Selection of News. *Journalism Bulletin*, *27*, 383–390. doi:10.1177/107769905002700403.

Wu, L., & Liu, H. (2018). Tracing Fake-News Footprints: Characterizing Social Media Messages by How They Propagate. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining* WSDM '18 (pp. 637–645). New York, NY, USA: ACM.

Yang, F., Liu, Y., Yu, X., & Yang, M. (2012). Automatic Detection of Rumor on Sina Weibo. In *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics* MDS '12 (pp. 13:1–13:7). New York, NY, USA: ACM. doi:10.1145/2350190.2350203.

Yu, F., Liu, Q., Wu, S., Wang, L., & Tan, T. (2019). Attention-based convolutional approach for misinformation identification from massive and noisy microblog posts. *Computers & Security*, *83*, 106 – 121. doi:https://doi.org/10.1016/j.cose.2019.02.003.

Zhou, X., & Zafarani, R. (2018). Fake news: A survey of research, detection methods, and opportunities. *CoRR*, *abs/1812.00315*. URL: http://arxiv.org/abs/1812.00315. arXiv:1812.00315.